Kubernetes and Cloud Native Associate - KCNA

Scheduling

Node Affinity

Welcome to this comprehensive guide on node affinity in Kubernetes. Node affinity allows you to control the placement of your pods by specifying rules about which nodes are eligible for scheduling. While traditional node selectors provided basic control, node affinity offers advanced scheduling features with flexible operators and expressions.

Simple Node Selector

Before exploring node affinity, consider a simple node selector that schedules a pod on a node labeled with size "Large":

```
apiVersion:
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
    - name: data-processor
      image: data-processor
  nodeSelector:
    size: Large
```

Using Node Affinity

Node affinity uses a similar underlying concept but allows for more advanced expressions. The following example demonstrates how to schedule a pod on a node with a label `size` whose value is in the specified list:

```
apiVersion:
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
    - name: data-processor
      image: data-processor
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: size
                operator: In
                values:
                  - Large
```

Understanding the Configuration

In this configuration:

- The `affinity` block is defined under the pod `spec`.
- `nodeAffinity` specifies the criteria used for node scheduling.
- The field `requiredDuringSchedulingIgnoredDuringExecution` indicates a mandatory requirement for scheduling; if no node meets the criteria, the pod is not scheduled.
- `nodeSelectorTerms` holds an array of conditions—in this case, ensuring that the node label `size` must have a value included in

the specified list.

To allow flexibility—for example, if the pod can also run on a "Medium" node—you simply add that value to the list:

```
apiVersion:
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
    - name: data-processor
      image: data-processor
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: size
                operator: In
                values:
                  - Large
                  - Medium
```

Alternatively, if you want to exclude nodes labeled as "Small", you can use the `NotIn` operator:

```
apiVersion:
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
    - name: data-processor
      image: data-processor
  affinity:
```

```
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: size
                operator: NotIn
                values:
                  - Small
```

The `Exists` operator offers another approach. Rather than comparing against specific values, it checks for the presence of the label. This example schedules the pod on any node where the `size` label is defined:

```
apiVersion:
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
    - name: data-processor
      image: data-processor
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: size
                operator: Exists
```

For more details on the available operators, refer to the **Kubernetes documentation**.

Behavior and Lifecycle of Node Affinity

When a pod is created, the Kubernetes scheduler evaluates its node affinity rules to determine the node on which to schedule the pod. However, several scenarios can occur if node conditions change over time.

Node Affinity Types

There are two primary types of node affinity currently supported:

- **requiredDuringSchedulingIgnoredDuringExecution**:
  The scheduler enforces that the pod be placed on a node that satisfies the affinity rules. If no matching node is available, the pod remains unscheduled. Once the pod is running, changes to node labels do not impact the running pod.

- **preferredDuringSchedulingIgnoredDuringExecution**:
  The scheduler attempts to honor the specified node affinity rules. If a matching node is not found, the pod can be scheduled on a non-matching node. Similarly, node label changes after scheduling are ignored.

Upcoming Enhancements

Future releases of Kubernetes plan to introduce additional affinity types that enforce rules during both scheduling and execution:

- requiredDuringSchedulingRequiredDuringExecution
- preferredDuringSchedulingRequiredDuringExecution

Scheduling vs. Execution

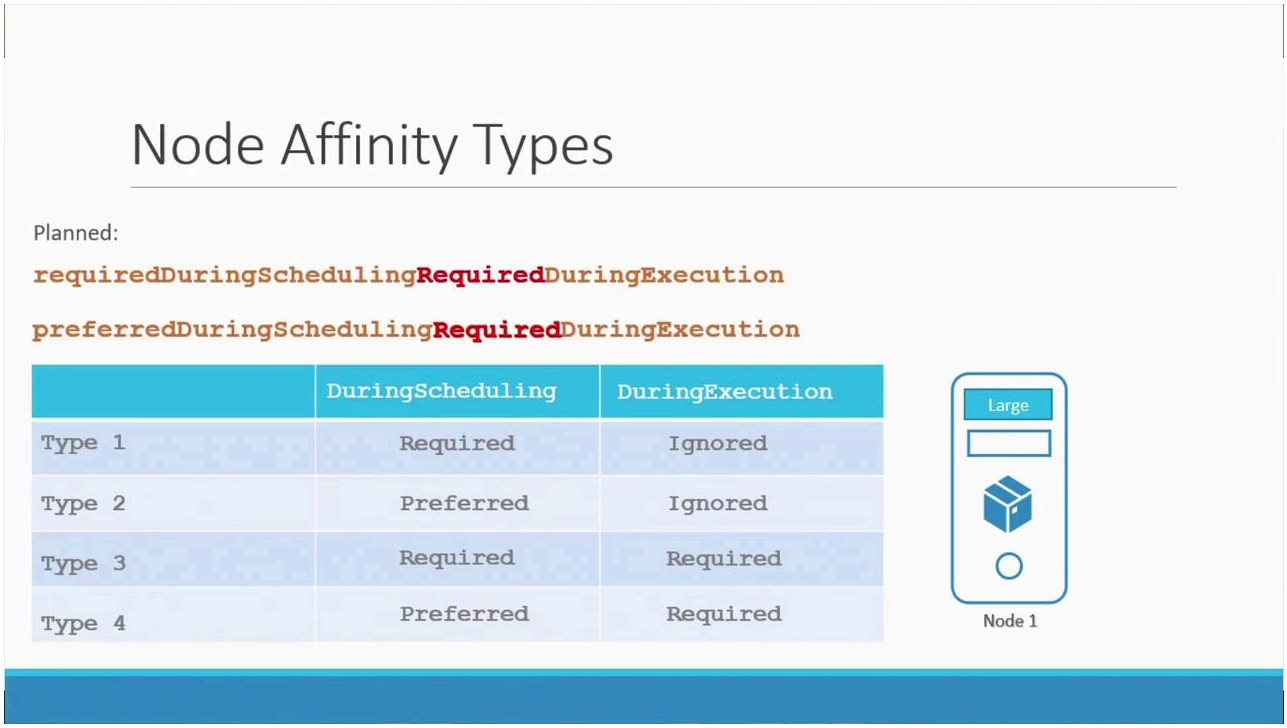
Node affinity rules are applied during two key phases of a pod's lifecycle:

1. **During Scheduling:**
   At the time of pod creation, the scheduler evaluates the node affinity rules to determine an appropriate node. If using the required type and no matching node is found (for example, if a node is missing the expected label "Large"), the pod will not be scheduled.

2. **During Execution:**
   Once the pod is running, changes in node labels are typically ignored for the

current affinity types ("ignored during execution"). However, with forthcoming execution-enforced rules, pods might be evicted if the node subsequently fails to meet the affinity criteria.

Consider this scenario:
A pod is scheduled on a node with the label `size=Large`. If an administrator later removes this label, the pod continues to run under the current behavior. Future implementations with the "required during execution" option could result in pod eviction.



Conclusion

In this guide, we broke down the core components of node affinity and demonstrated how various operators and affinity types influence pod scheduling and execution in Kubernetes. Understanding and leveraging these advanced scheduling capabilities allows you to optimize node usage and ensure that pods are placed on nodes that best meet your application requirements.

For further reading and advanced configuration options, be sure to check out the **Kubernetes Documentation**.

Watch Video

Watch video content

Previous

Node Selectors

Next

Taints and Tolerations vs Node Affinity